WEEK 3

This week I managed to read up about witness and public inputs in ZKP as previously mentioned. My understanding is that the witness is the private information that only the prover knows and the public inputs are non-secret values known to both the verifier and prover. Basically, in ZKP, given public input x and claimed output y, we are proving there exists a witness w such that it satisfies this relation $C(x, w) = y$ where C is the arithmetic circuit or computation being checked.
I have read briefly from PAZK Sections 10.1 to 11.2. It took me some time to understand PIOPs and the related concepts. I have summarized my understanding of the suggested questions below:

What is a PIOP, and what is a Polynomial Commitment?

PIOP is an interactive proof where instead of sending ordinary text to the verifier, these messages are represented as polynomials. Moreover, the verifier does not need to know all the coefficients of the entire polynomial but instead only require its value at a random point. It is useful as now the polynomial can be as large as the whole R1CS instance, and the verifier only needs to check a small number of random polynomial evaluations.

Polynomial commitment lets the prover commit to a polynomial first and later prove evaluations of that polynomial at chosen points. This lets the verifier confirm that the prover's claimed evaluation really belongs to the committed polynomial of the required degree.

How do you implement the Univariate Sum-Check PIOP and the Univariate Zero-Test PIOP?

The Univariate Sum-check PIOP checks whether the sum of the polynomial values over a set H adds up to zero. So, the prover will start with the polynomial P and build the vanishing polynomial for a set H. Then the prover divides P by the vanishing polynomial to get the quotient and remainder. These are used to create the helper polynomials, which the prover commits to. The verifier then chooses a random point and asks the prover for the polynomial's values at that point. Then the verifier checks whether the polynomial identity holds at the random point.

For the Univariate Zero-Test PIOP, it basically tests whether the polynomial is zero on every point in a set H. The prover takes the polynomial P and divides it by the vanishing polynomial of H. Then, the prover commits to the quotient polynomial. The verifier picks a random point r and checks whether $P(r)$ is equal to the quotient evaluated at r multiplied by the vanishing polynomial evaluated at r. If this equality holds then the verifier can accept with high probability that P is zero at all points.

The workflow of R1CS-SAT.

R1CS-SAT is about checking whether there exists an assignment of values that satisfies all the constraints of a computation. The main idea is taking the computation and converting it into an arithmetic circuit which will be converted to R1CS constraints. The R1CS instance is then represented using polynomials and PIOP checks, such as zero test and sum-check, are used to verify the required polynomial relations. After a few evaluations at random points, if the polynomials checks pass, the verifier would be convinced that the computation is done correctly with high probability.

How does a Brakedown Polynomial Commitments work?

It is a type of polynomial commitment that makes the prover computation very fast, improving the prover time from $O(nlogn)\ to\ O(n)$. The basic idea is that instead of treating the polynomial as one long list of coefficients, the prover breaks it down into rows like a matrix. Each row is then encoded using an error-correcting code and the prover commits to this encoded matrix. When the verifier asks for a value, the prover sends the evaluation of some linear combination of row and column information. This allows the verifier to be convinced that the evaluation comes from the committed polynomial while keeping the prover's computation very efficient.

What is Zero-knowledge?

It basically means that the verifier becomes convinced that the statement is true but learns nothing about the secret witness. That means that the proof only shows that prover has a valid secret but it does not reveal what the secret is.